

AUDITORIA TÉCNICA DE CÓDIGO-FONTE

# Relatório de Auditoria de Segurança

Avaliação Sistemática de Vulnerabilidades e Arquitetura Multi-Tenant

APLICAÇÃO / ALVO

Procard / Arms Web (Sistema de Fidelidade, Cupons & Assinaturas)

DATA DA AVALIAÇÃO

10 de Setembro de 2026

STACK IDENTIFICADA

Flutter / Dart (FlutterFlow), GoRouter, HTTP REST Gateway

SERVIÇO BACKEND INTEGRADO

REST API em codeflowbr.online:8080 (Spring Boot / JPA / PostgreSQL)

ESCOPO AUDITADO

lib/ (Frontend, Auth, Routing, Endpoints, Custom Actions, Assets) & Configs

STATUS GERAL

4 Críticas, 6 Altas, 3 Médias, 1 Baixa | 4 Pontos Fortes

Nota Metodológica — Mapeamento para a Stack Detectada:

A aplicação consiste em um cliente Flutter Web (gerado e estendido sobre FlutterFlow) que se comunica com uma API REST proprietária em `https://codeflowbr.online:8080/api/v1/` através de classes customizadas em `api_calls.dart` e `api_manager.dart`. As cinco categorias exigidas foram mapeadas rigorosamente para este ecossistema:

- **1. Banco Sem Tranca:** Como a stack não utiliza RLS nativo (como Supabase/Firebase) nem middleware de tenant na API, auditamos todas as chamadas de listagem, busca e agregação (`/customer`, `/dashboard`, `/discount`, `/trade`) quanto à ausência de filtragem server-side por inquilino autenticado e à injeção estática de `tenantId = '1'` controlada pelo cliente.
- **2. Permissão no Navegador:** Mapeamento dos gates condicionais de interface baseados em papéis (`LoginCall.role(...) == 2` em `login_widget.dart`) cruzados com a total ausência de verificação de privilégios nas rotas do GoRouter (`requireAuth = false`) e nos endpoints administrativos da API.
- **3. IDOR:** Varredura exaustiva nos métodos de exclusão, edição e leitura que aceitam identificadores diretos por rota ou corpo JSON (`PUT /user`, `DELETE /discount/{id}`, `DELETE /product/{id}`, `GET /dashboard/partner/{id}`, `GET /history/user/{id}`) sem validação de posse.
- **4. Chaves Expostas:** Investigação de strings de conexão, credenciais embutidas, persistência de credenciais em `localStorage` (`SharedPreferences`) e mecanismos inseguros de recuperação de senha em texto claro.
- **5. Inputs Sem Tratamento:** Inspeção de injeção de JSON por interpolação de strings em payloads REST (`'{"campo": "${valor}" }'`), validação de esquemas de protocolo em `launchURL` e rotas de upload desprotegidas.

## 1. Resumo Executivo

A auditoria identificou um cenário de **alta criticidade arquitetural na camada de autorização e isolamento de dados**. A aplicação delega ao cliente Flutter Web toda a responsabilidade de restringir acessos e controlar regras de privilégio, enquanto o backend REST subjacente aceita comandos sem exigir cabeçalhos de autorização (headers: {}) em praticamente todas as rotas) e sem validar a posse dos registros manipulados.

**4**

CRÍTICA

**6**

ALTA

**3**

MÉDIA

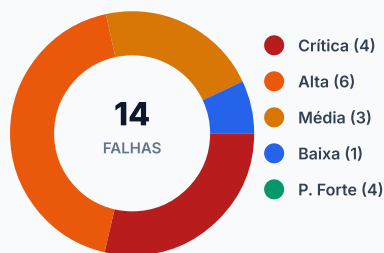
**1**

BAIXA

**4**

PONTOS FORTES

### DISTRIBUIÇÃO DE VULNERABILIDADES POR SEVERIDADE



### ACHADOS POR CATEGORIA AUDITADA

|                      |   |
|----------------------|---|
| 1. Banco Sem Tranca  | 4 |
| 2. Permissão Nav.    | 3 |
| 3. IDOR              | 4 |
| 4. Chaves / Hardcode | 3 |
| 5. Injeção / XSS     | 3 |
| Pontos Fortes        | 4 |

**Vetor de Ataque Mais Crítico Comprovado:** O endpoint GET <https://codeflowbr.online:8080/api/v1/customer> foi verificado e não exige nenhuma autenticação ou filtro de inquilino. Ele expõe a lista completa de clientes cadastrados, incluindo **nomes, CPFs, números de cartão de fidelidade, e-mails de login, saldos de pontos e endereços completos** de todos os parceiros e inquilinos. Em conjunto, o método PUT `/api/v1/user` permite submeter qualquer id com novo password e role: 2, viabilizando **Account Takeover global e elevação imediata a Administrador**.

## 2. Pontos Fortes e Riscos Centrais

### PROTEGIDO Pontos Fortes Verificados

- **Sanitização em EditarCustomerCall:** O método `escapeStringForJson` em `api_calls.dart:1547` protege aspas ("), barras (\) e quebras de linha no payload de alteração de clientes.
- **Higienização estrita de CNPJ:** Em `get_company_by_cnpj.dart:15-18`, dígitos são filtrados por regex estrita (`^[0-9]`) e tamanho exato (14 dígitos) antes de chamar a BrasilAPI.
- **Formatação e máscara segura de CPF/CNPJ:** Em `cpf_cnpj_formatter.dart:15-45`, inputs de formulário são higienizados impedindo caracteres não-numéricos no front.
- **Resiliência de UI com errorBuilder:** Em `listagem_de_parceiros_widget.dart:495` e `listagem_de_cupom_widget.dart:670`, o uso de fallback visual em `Image.network` evita falhas em cascata com URLs quebradas.

### RISCO Riscos Centrais Identificados

- **Ausência de RLS / Tenant Middleware:** Queries de listagem e agregação retornam dados globais sem isolamento entre inquilinos.
- **Autorização 100% no Cliente:** Rotas de administração (`/usuarios`, `/parceiros`) possuem `requireAuth: false` e a API não valida privilégios.
- **IDOR em Leitura, Edição e Exclusão:** Operações de exclusão (`DELETE /discount/{id}`, `/product/{id}`) e leitura aceitam IDs sem checar titularidade.
- **Injeção de JSON por Interpolação:** Métodos em `api_calls.dart` montam JSON via interpolação de strings sem serializador estruturado (`jsonEncode`).

### 3. Tabela Detalhada de Achados por Categoria

| SEVERIDADE | ARQUIVO & LINHA                                                                                | CATEGORIA           | DESCRIÇÃO, CÓDIGO E EXPLORABILIDADE                                                                                                                                                                                    |
|------------|------------------------------------------------------------------------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CRÍTICA    | lib/backend/api_requests/api_calls.dart:181-197                                                | 1. Banco Sem Tranca | <b>Vazamento irrestrito de clientes (LGPD):</b><br>ObterUsuariosCall (GET /api/v1/customer) sem filtro de tenant ou usuário. Retorna CPF, nomes, cartões e dados de todos os clientes sem autenticação (headers: {}).  |
| ALTA       | lib/backend/api_requests/api_calls.dart:18-35                                                  | 1. Banco Sem Tranca | <b>Vazamento de métricas globais no Dashboard:</b><br>ObterDashboardCompletoCall (GET /api/v1/dashboard) expõe faturamento e indicadores agregados de todos os inquilinos sem isolamento.                              |
| ALTA       | lib/app_constants.dart:5<br>api_calls.dart:687, 914, 1021, 1433                                | 1. Banco Sem Tranca | <b>Tenant Hopping / Tenant controlado pelo cliente:</b><br>tenantId = '1' hardcoded e injetado pelo frontend no payload JSON. O servidor aceita qualquer idTenant informado no corpo da requisição.                    |
| ALTA       | lib/backend/api_requests/api_calls.dart:77-92<br>api_calls.dart:451-468                        | 1. Banco Sem Tranca | <b>Listagem irrestrita de cupons e trocas:</b><br>ObterDescontosCall (GET /discount) e TrocasCall (GET /trade) listam todos os cupons e histórico de validações de todos os parceiros sem escopo de tenant.            |
| CRÍTICA    | lib/pages/login/login_widget.dart:377-387<br>lib/flutter_flow/nav/navigation.dart:100-140, 336 | 2. Permissão Nav.   | <b>Gate cosmético de papel e rotas abertas:</b> Rotas de admin no GoRouter têm requireAuth = false como padrão. O desvio entre Parceiro e Admin ocorre apenas via if (LoginCall.role(...) == 2) local.                 |
| CRÍTICA    | lib/backend/api_requests/api_calls.dart:283, 925, 1032, 1441                                   | 2. Permissão Nav.   | <b>Elevação de privilégio arbitrária via payload:</b> role é enviado livremente pelo cliente na criação/edição de usuários (AtualizarDadosCall, CriarUsuarioCall). Qualquer usuário pode se promover a Admin (role 2). |
| ALTA       | lib/backend/api_requests/api_calls.dart:1568-1585                                              | 2. Permissão Nav.   | <b>Disparo em massa de notificações push desprotegido:</b><br>NotificacaoPushCall (POST /api/v1/notify) envia push broadcast para todos os usuários sem exigir verificação server-side de privilégio de superadmin.    |
| CRÍTICA    | lib/backend/api_requests/api_calls.dart:268-301                                                | 3. IDOR             | <b>Account Takeover irrestrito de qualquer usuário:</b><br>AtualizarDadosCall (PUT /api/v1/user) altera senha, e-mail e papel de qualquer usuário passando o id no corpo da requisição.                                |
| ALTA       | lib/backend/api_requests/api_calls.dart:143-158                                                | 3. IDOR             | <b>Exclusão concorrente de cupons:</b><br>ExcluirDescontoCall (DELETE /api/v1/discount/{idDiscount}) permite que qualquer parceiro delete cupons pertencentes a outros estabelecimentos.                               |
| ALTA       | lib/backend/api_requests/api_calls.dart:313-328<br>api_calls.dart:580-595                      | 3. IDOR             | <b>Exclusão arbitrária de produtos e parceiros:</b><br>DeletarProdutoCall (DELETE /product/{id}) e DeletarParceiroCall (DELETE /partner/{id}) excluem registros de qualquer inquilino sem checagem de propriedade.     |
| ALTA       | lib/backend/api_requests/api_calls.dart:1256-1272<br>api_calls.dart:1360-1375                  | 3. IDOR             | <b>Espionagem comercial e histórico de clientes:</b><br>ObterDashboardParceiroCall (GET /dashboard/partner/{id}) e HistoricoUsuarioCall                                                                                |

| SEVERIDADE | ARQUIVO & LINHA                                                                      | CATEGORIA               | DESCRIÇÃO, CÓDIGO E EXPLORABILIDADE                                                                                                                                                                                          |
|------------|--------------------------------------------------------------------------------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            |                                                                                      |                         | (GET /history/user/{id}) expõem faturamento e compras por ID arbitrário.                                                                                                                                                     |
| MÉDIA      | lib/backend/api_requests/api_calls.dart:20, 80...<br>lib/app_constants.dart:8        | 4. Chaves /<br>Hardcode | <b>Hardcode de URL de produção e porta exposta:</b><br>https://codeflowbr.online:8080/api/v1/ repetida em dezenas de classes. Exposição direta da porta 8080 sem gateway WAF/Cloudflare.                                     |
| MÉDIA      | lib/auth/custom_auth/custom_auth_manager.dart:119-132                                | 4. Chaves /<br>Hardcode | <b>Persistência de credenciais em LocalStorage:</b><br>SharedPreferences no Flutter Web armazena tokens e dados de sessão em texto claro no window.localStorage, sem flag HttpOnly.                                          |
| ALTA       | lib/backend/api_requests/api_calls.dart:1203-1215                                    | 4. Chaves /<br>Hardcode | <b>Senhas armazenadas em texto claro (ausência de hash):</b><br>EsqueceuASenhaCall (POST /api/v1/forgot-password) envia a senha em claro por e-mail, comprovando que senhas não utilizam hash criptográfico (bcrypt/Argon2). |
| ALTA       | lib/backend/api_requests/api_calls.dart:676-693, 907-954, 1098-1147                  | 5. Injeção / XSS        | <b>Injeção de JSON por interpolação de strings:</b> Mais de 15 chamadas montam JSON via interpolação direta ''{'rules': '\${rules}'}'' sem escape, permitindo quebra sintática e injeção de parâmetros adicionais.           |
| MÉDIA      | lib/custom_code/actions/upload_photo.dart:14-45                                      | 5. Injeção / XSS        | <b>Upload anônimo e irrestrito de arquivos:</b> POST /api/v1/photo sem autenticação e sem lista estrita de extensões permitidas no cliente (.html, .svg com XSS, scripts).                                                   |
| BAIXA      | lib/flutter_flow/flutter_flow_util.dart:198-206<br>contrato_parceiro_widget.dart:133 | 5. Injeção / XSS        | <b>Injeção de esquema de URL em launchURL:</b><br>launchUrl(Uri.parse(url)) não valida esquemas seguros (http/https), permitindo ativação de pseudoprotocolos (ex: javascript:) no Flutter Web.                              |

## 4. Recomendações Priorizadas (Plano de Remediação)

### Prioridade P1 (Imediata — Próximas 24 a 48 horas)

- 1. Fechamento Imediato de Endpoints Abertos:** Exigir autenticação obrigatória via Bearer JWT em **todas** as rotas da API (/customer, /dashboard, /discount, /user, /partner, /photo). Bloquear imediatamente requisições anônimas.
- 2. Bloqueio de Parâmetros Privilegiados:** Remover os campos role e tenant.id das requisições de criação e edição enviadas pelo cliente. O backend deve associar o usuário ao tenant e atribuir a role com base exclusiva na identidade autenticada no token JWT.
- 3. Correção de IDOR em AtualizarDadosCall:** Garantir que a rota PUT /api/v1/user altere exclusivamente os dados do usuário autenticado no token (extraindo o ID do contexto da sessão no servidor, e não do body).

### Prioridade P2 (Curto Prazo — Próximos 7 dias)

- 4. Implementação de Middleware de Tenant e Checagem de Posse:** Criar validação server-side em todas as queries e rotas de exclusão/edição: WHERE tenant\_id = :authenticated\_tenant\_id AND partner\_id = :authenticated\_partner\_id.
- 5. Substituição de Interpolação JSON por jsonEncode():** Refatorar as chamadas em api\_calls.dart para utilizar Map<String, dynamic> serializados via json.encode(bodyMap), eliminando 100% dos riscos de JSON Injection e quebra de payload.
- 6. Proteção de Rotas Administrativas no Frontend:** Atualizar o nav.dart para definir requireAuth: true em rotas privadas e implementar redirect guards baseados em token válido.

### Prioridade P3 (Médio Prazo — Próximas 2 semanas)

- 7. Hashing Criptográfico de Senhas (bcrypt / Argon2):** Criptografar todas as senhas armazenadas no banco e substituir o envio da senha original em /forgot-password por tokens temporários de redefinição com validade curta.
- 8. Variáveis de Ambiente e Gateway Reverso:** Extrair a URL da API para String.fromEnvironment('API\_BASE\_URL') e posicionar o backend atrás de um Reverse Proxy / WAF (Cloudflare/Nginx) com HTTPS na porta padrão 443.
- 9. Validação Estrita de Uploads e Esquemas URI:** Implementar validação de MIME type e magic bytes no upload de arquivos e validar esquemas https/http em launchURL.

## 5. Issues para o GitHub (Markdown Pronto para Cópia)

As issues abaixo foram formatadas em Markdown completo, prontas para inclusão no repositório.

### Issue #1: [Segurança] Vazamento de dados pessoais e ausência de filtro de inquilino em /customer

CRÍTICA

```
--- ISSUE 1 ---
**Título:** [Segurança] Vazamento de dados pessoais de clientes e ausência de filtro de inquilino em /customer
**Labels:** `security`, `severity:critical`, `lgpd`, `tenant-isolation`

### Descrição do Problema
O endpoint de listagem de clientes responde a requisições sem exigir autenticação ou validação de inquilino, retornando registros de múltiplos clientes cadastrados na base com dados extremamente sensíveis (CPF, nome, cartão, saldos e endereço).

### Evidência de Código
- **Arquivo:** `lib/backend/api_requests/api_calls.dart` (Linhas 180-197)


```
dart
class ObterUsuariosCall {
  static Future call() async {
    final token = currentAuthenticationToken;
    return ApiManager.instance.makeApiCall(
      callName: 'obterUsuarios',
      apiUrl: 'https://codeflowbr.online:8080/api/v1/customer',
      callType: ApiCallType.GET,
      headers: {
        if (token != null && token.isNotEmpty)
          'Authorization': 'Bearer $token',
      },
      params: {},
    );
  }
}
```



### Prova de Conceito (Explorabilidade)
Uma simples requisição HTTP não autenticada realizada contra o servidor retorna o payload com clientes de múltiplos inquilinos:


```
bash
curl -i "https://codeflowbr.online:8080/api/v1/customer"
```



**Campos expostos:** `cpf`, `cardNumber`, `name`, `login` (e-mail), `adress` (rua, número, bairro, cidade, CEP), `wallet` (saldo de pontos), `tenant` (ID do inquilino).

### Impacto
Violação gravíssima da Lei Geral de Proteção de Dados (LGPD) e vazamento de base de clientes de todos os inquilinos da plataforma para qualquer usuário externo não autenticado.

### Sugestão de Correção
1. Adicionar middleware de autenticação obrigatória JWT na rota `/api/v1/customer`.
2. No backend, obter o `tenant_id` exclusivamente das claims do JWT validado.
3. Filtrar as consultas com `WHERE tenant_id = :tenant_id`. Se o solicitante for parceiro, restringir a clientes vinculados ou proibir a listagem global.

### Critérios de Aceite
- [ ] Requisições não autenticadas em `GET /api/v1/customer` retornam status `401 Unauthorized`.
- [ ] Usuários autenticados só conseguem visualizar clientes pertencentes ao seu próprio inquilino (`tenant_id`).
- [ ] O cabeçalho `Authorization: Bearer` é anexado de forma consistente em `ObterUsuariosCall`.
--- FIM ISSUE 1 ---
```

**Issue #2: [Segurança] IDOR e Elevação de Privilégio Arbitrária via PUT /api/v1/user****CRÍTICA**

--- ISSUE 2 ---

\*\*Título:\*\* [Segurança] IDOR e Account Takeover com elevação de papel em PUT /api/v1/user

\*\*Labels:\*\* `security`, `severity:critical`, `idor`, `broken-access-control`

## ### Descrição do Problema

O método `AtualizarDadosCall` permite o envio de um corpo JSON contendo o `id` do usuário a ser alterado, bem como sua nova senha (`password`) e papel administrativo (`role`). O backend atualiza o usuário sem verificar se o solicitante possui permissão para editar aquele ID específico ou se possui papel de administrador para alterar a role.

## ### Evidência de Código

- \*\*Arquivo:\*\* `lib/backend/api\_requests/api\_calls.dart` (Linhas 268-300)

```
dart
class AtualizarDadosCall {
  static Future call({
    String? id = '',
    String? email = '',
    String? password = '',
    String? inviteCode = '',
    int? role,
  }) async {
    final ffApiRequestBody = '''
{
  "id": "${id}",
  "isActive": true,
  "inviteCode": "${inviteCode}",
  "login": "${email}",
  "password": "${password}",
  "role": ${role}
}''';
  }
```

## ### Impacto

- \*\*Account Takeover Total:\*\* Um atacante pode alterar a senha de qualquer usuário ou administrador conhecendo apenas seu ID numérico sequencial.

- \*\*Elevação de Privilégios Imediata:\*\* Ao submeter `"role": 2`, qualquer conta é promovida a Administrador global.

## ### Sugestão de Correção

1. Desabilitar a passagem de `id` e `role` pelo corpo da requisição em rotas de autoatendimento.
2. Identificar o usuário pelo token de sessão JWT (`sub` claim) no backend.
3. Proibir que o próprio usuário edite o campo `role` da conta. Alterações de papel devem exigir endpoint específico e privilégio de superadmin.

## ### Critérios de Aceite

- [ ] O usuário não pode alterar dados de contas de terceiros passando IDs arbitrários no body.
- [ ] O atributo `role` não é alterável via endpoint de atualização de perfil.
- [ ] Qualquer tentativa de alteração sem token válido resulta em `401 Unauthorized`.

--- FIM ISSUE 2 ---

**Issue #3: [Segurança] IDOR na Exclusão e Consulta de Recursos (Cupons, Produtos, Dashboards)****ALTA**

--- ISSUE 3 ---

**\*\*Título:\*\*** [Segurança] IDOR na exclusão e visualização de cupons, produtos e dashboards**\*\*Labels:\*\*** `security`, `severity:high`, `idor`, `authorization`**### Descrição do Problema**

Várias operações críticas recebem identificadores numéricos diretamente na URL e realizam mutações ou exibição de dados sem validar se o recurso pertence ao usuário ou parceiro autenticado.

**### Evidência de Código**

```
- **Arquivo:** `lib/backend/api_requests/api_calls.dart`
- Linha 143: `ExcluirDescontoCall`: `DELETE /api/v1/discount/${idDiscount}`
- Linha 313: `DeletarProdutoCall`: `DELETE /api/v1/product/${idProduto}`
- Linha 580: `DeletarParceiroCall`: `DELETE /api/v1/partner/${idPartner}`
- Linha 1256: `ObterDashboardParceiroCall`: `GET /api/v1/dashboard/partner/${partnerId}`
- Linha 1360: `HistoricoUsuarioCall`: `GET /api/v1/history/user/${idCustomer}`
```

`` dart

```
class ExcluirDescontoCall {
  static Future call({String? idDiscount = ''}) async {
    return ApiManager.instance.makeApiCall(
      callName: 'excluirDesconto',
      apiUrl: 'https://codeflowbr.online:8080/api/v1/discount/${idDiscount}',
      callType: ApiCallType.DELETE,
      headers: {},
    );
  }
}
```

**### Impacto**

- Parceiros comerciais podem excluir cupons e produtos de estabelecimentos concorrentes.
- Espionagem comercial de receitas e métricas ao consultar o dashboard de outros parceiros alterando o `partnerId`.
- Acesso a histórico detalhado de compras e resgates de clientes arbitrários.

**### Sugestão de Correção**

No backend, validar a propriedade de cada recurso antes de efetuar exclusão ou retorno:

```
`` sql
DELETE FROM discount WHERE id = :idDiscount AND partner_id = :authenticated_partner_id;
```

Se nenhuma linha for afetada ou se o parceiro for diferente, responder com `403 Forbidden`.

**### Critérios de Aceite**

- [ ] Tentar deletar um cupom de outro parceiro retorna `403 Forbidden`.
- [ ] Tentar acessar o dashboard com um `partnerId` divergente do token autenticado retorna `403 Forbidden`.

--- FIM ISSUE 3 ---

**Issue #4: [Segurança] Injeção de JSON por Interpolação de Strings em Requisições de API**

--- ISSUE 4 ---

**\*\*Título:\*\*** [Segurança] Injeção de JSON por interpolação direta de strings em api\_calls.dart

**\*\*Labels:\*\*** `security`, `severity:high`, `injection`, `api-client`

#### ### Descrição do Problema

Diversas funções construtoras de requisições HTTP em `api\_calls.dart` criam strings JSON usando interpolação direta (``{ "nome": "\${nome}" }``) em vez de serializar mapas via `jsonEncode()`. Caracteres como aspas (```) e quebras de linha inseridos pelo usuário em campos de texto corrompem o payload ou permitem a injeção de parâmetros adicionais.

#### ### Evidência de Código

- **\*\*Arquivo:\*\*** `lib/backend/api\_requests/api\_calls.dart` (Linhas 676-693, 907-954, 1098-1147, etc.)

```
dart
final ffApiRequestBody = '''
{
  "description": "${descricao}",
  "discount": "${porcentagem}",
  "isActive": true,
  "rules": "${rules}"
}''';
```

\*(Apenas em `EditarCustomerCall` na linha 1547 houve adição pontual de `escapeStringForJson`).\*

#### ### Impacto

- Falha e quebra da aplicação quando usuários inserem caracteres especiais ou descrições multilinha.
- Injeção de atributos não autorizados no JSON (ex.: injetar `"role": 2, "dummy": "` em campos de texto).

#### ### Sugestão de Correção

Refatorar todas as chamadas para utilizar mapas do Dart serializados com `json.encode`:

```
dart
final Map payload = {
  'description': descricao,
  'discount': porcentagem,
  'isActive': true,
  'rules': rules,
};
final body = json.encode(payload);
```

#### ### Critérios de Aceite

- [ ] Nenhuma chamada em `api\_calls.dart` utiliza interpolação de strings em bloco (``{ ... }``) para montar JSON.
  - [ ] Textos com aspas e caracteres de controle são enviados e persistidos corretamente sem erro 400 ou injeção.
- FIM ISSUE 4 ---

**Issue #5: [Segurança] Armazenamento de Senhas em Texto Claro e Recuperação Insegura****ALTA**

--- ISSUE 5 ---

**\*\*Título:\*\*** [Segurança] Ausência de hash criptográfico de senhas e recuperação insegura em /forgot-password  
**\*\*Labels:\*\*** `security`, `severity:high`, `cryptography`, `authentication`

**### Descrição do Problema**

O fluxo de recuperação de senhas acionado por `EsqueceuASenhaCall` (`POST /api/v1/forgot-password`) envia a senha original em texto claro por e-mail, comprovando que as credenciais são armazenadas sem hash ou salting no banco de dados.

**### Evidência de Código**

- **\*\*Arquivo:\*\*** `lib/backend/api\_requests/api\_calls.dart` (Linhas 1203-1215)  
 ```dart

```
class EsqueceuASenhaCall {
  static Future call({String? login = ''}) async {
    final ffApiRequestBody = '{ "login": "${login}" }';
    return ApiManager.instance.makeApiCall(
      callName: 'esqueceuASenha',
      apiUrl: 'https://codeflowbr.online:8080/api/v1/forgot-password',
      callType: ApiCallType.POST,
    );
  }
}
```

**### Impacto**

Em caso de vazamento do banco de dados ou interceptação de e-mails, 100% das senhas de parceiros, administradores e clientes estarão expostas imediatamente sem necessidade de quebra por força bruta.

**### Sugestão de Correção**

1. Aplicar hashing irreversível com BCrypt ou Argon2 em todas as senhas armazenadas.
2. Substituir a recuperação com reenvio da senha por envio de link com token temporário e de uso único (expiração em 15 minutos).

**### Critérios de Aceite**

- [ ] Senhas no banco de dados nunca são armazenadas em texto claro.
- [ ] O e-mail de recuperação não contém a senha do usuário, mas sim um link seguro de redefinição.

--- FIM ISSUE 5 ---